

improving

July 31, 2025

1 Improving the MNIST Digit Prediction Model

The model we previously trained (in `exploration.ipynb`) did not perform well on custom data. As such, we are going to try to make a better model here.

```
[1]: from pathlib import Path
import random
import typing

import matplotlib.pyplot as plt
from matplotlib.figure import Figure
from matplotlib.axes import Axes
import numpy as np
import seaborn as sns
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch import Tensor
from torchvision import transforms, datasets

import mnist
from mnist.utils.plot import PltAxes, PltFigure
```

```
[2]: if torch.cuda.is_available():
    device = 'cuda'
else:
    device = 'cpu'
print(f'Using {device}')
```

Using cuda

We load the validation dataset so as to not debug on the training data that the model has already seen.

```
[3]: data_root = Path('../data')
x_mean, x_std = (0.13066114485263824, 0.30810731649398804)
classes = list(map(str, range(10)))

val_transform = transforms.Compose([
```

```

        transforms.RandomAffine( # Random affine transformation placeholder. For
        ↪now there are no transformations applied.
            degrees=0,
            translate=(0, 0),
            scale=(1, 1),
            shear=0
        ),
        transforms.ToTensor(),
        transforms.Normalize((x_mean,), (x_std,))
    ])
val_dataset = datasets.MNIST(root=data_root, train=False, download=True,
    ↪transform=val_transform)
val_loader = torch.utils.data.DataLoader(val_dataset, batch_size=64,
    ↪shuffle=True)

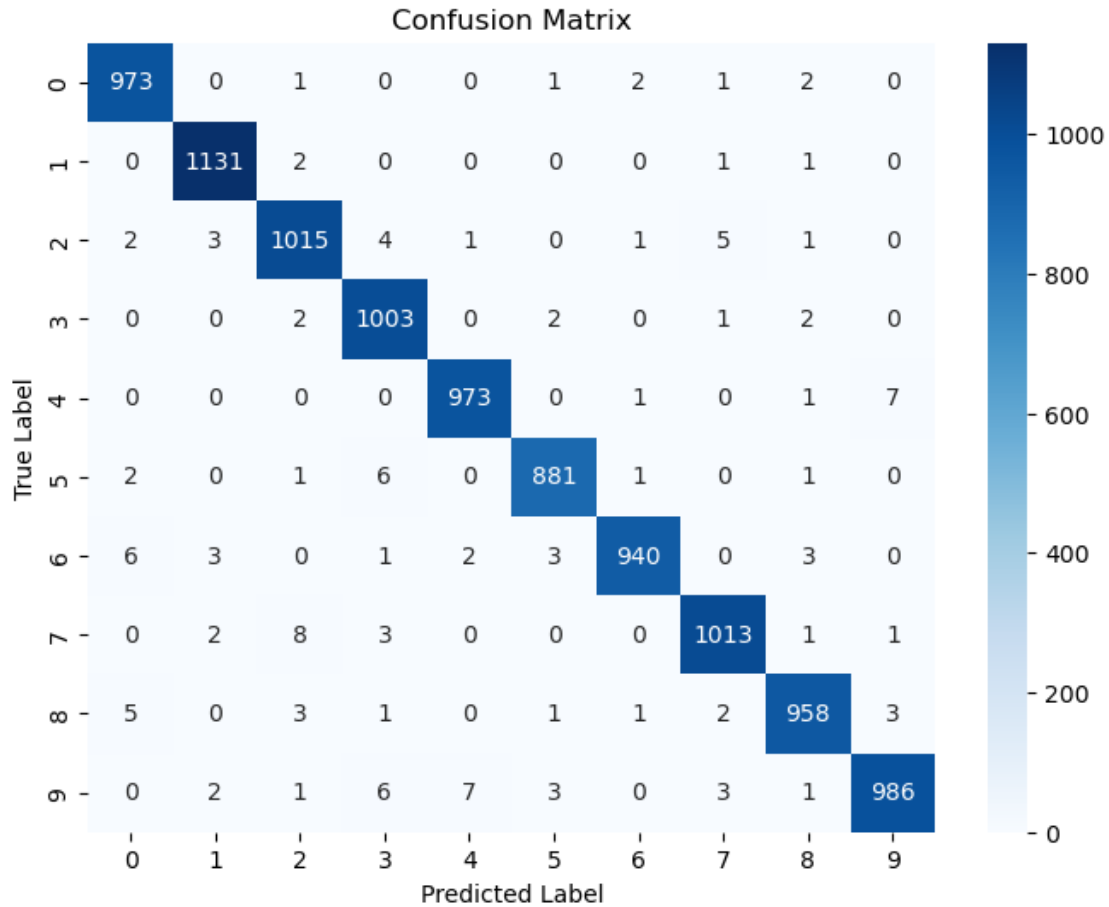
```

Shown below is a confusion matrix of the model's predictions (x-axis) compared with the true labels (y-axis) on the entire validation set. The strong diagonal shows that the model performs very well on this data, making only a small number of incorrect predictions.

```

[4]: cnn = mnist.model.cnn.CNN()
state_dict = torch.load('../models/torch/model2.pth', weights_only=True)
cnn.load_state_dict(state_dict)
cnn.to(device)
cm = mnist.utils.plot.generate_confusion_matrix(cnn, val_loader, device)
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=range(10),
    ↪yticklabels=range(10))
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.title("Confusion Matrix")
plt.show()

```

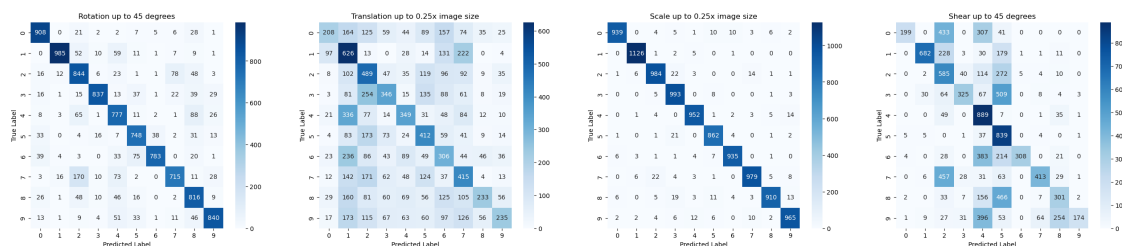


However, once transformations are applied, the model starts to deteriorate. It is worth noting that rotation and scaling do not have a large effect on the model's performance. For rotations, the majority of examples are estimated correctly with relatively small exceptions for numbers with similar features, like 2 and 7, and 4 and 8. The effects of scaling are similar but even less pronounced.

Translations, however, cause the model to struggle a great deal. This means the convolutional neural network (CNN) is not performing as it should, since CNNs are expected to detect features regardless of where they appear spatially in the image.

Shearing also poses a challenge for the network, but this could be due to the shearing distorting important features of the image. Further investigation will need to confirm this.

```
[ ]: fig: PltFigure
      axes: PltAxes
      fig, axes = plt.subplots(nrows=1, ncols=4, figsize=(32, 6))
      mnist.utils.plot.generate_transformation_confusion_matrices(cnn, data_root,
      ↪ x_mean, x_std, axes, device)
      plt.show()
```



To further inspect the translation problem, we will use Grad-CAM visualizations (using the [pytorch-grad-cam](#) library) to see what areas in the image the model is using as evidence to support the probabilities for each class.

To do this, we load the dataset without transforming it. Then we define the base transform (normalization) and the translation transform separately. The will both be manually applied and compared via Grad-CAM.

```
[6]: untransformed_val_dataset = datasets.MNIST(root=data_root, train=True,
        ↪download=True)

base_transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((x_mean,), (x_std,))
])

translate_transform = transforms.Compose([
    transforms.RandomAffine(
        degrees=0,
        translate=(0.25, 0.25),
        scale=(1, 1),
        shear=0
    ),
    transforms.ToTensor(),
    transforms.Normalize((x_mean,), (x_std,))
])
```

See below that the Grad-CAM visualizations often differ between the base and translated images. Ideally, a convolutional neural network should detect the same features in an image regardless of where the features are located. However, when the model's prediction is correct for the base image but incorrect for the translated image, the model is often detecting different features as a result of the translation.

As a caveat, sometimes when the prediction is correct for both the base and translated images the model is detecting slightly different features in the translated image. This somewhat discredits the above paragraph, since slight differences in detected features can still result in the same correct prediction. Additionally, the model sometimes incorrectly predicts the translated image even when it detects the same features as in the base image. This could demonstrate faulty behavior in the linear layers, rather than the convolutional layers.

```

[7]: from pytorch_grad_cam import GradCAM

N = 4
fig: PltFigure
axes: PltAxes
fig, axes = plt.subplots(nrows=2*N, ncols=11, figsize=(24, 24))

for i in range(0, 2*N, 2):
    pil_image, label = random.choice(untransformed_val_dataset)

    with GradCAM(model=cnn, target_layers=[cnn.conv_1_2]) as cam:
        cam.batch_size = 1

        # --- Base transform ---
        image: Tensor = base_transform(pil_image)
        image = image.unsqueeze(0)
        image = image.to(device)
        outputs = cnn(image)
        probabilities = F.softmax(outputs, dim=-1)
        predictions = torch.argmax(probabilities, dim=-1)

        mnist.utils.plot.generate_gradcam_class_plots(image, label,
        ↪ predictions, probabilities, cam, axes[i], base_image=True)

        # --- Translation transform ---
        image: Tensor = translate_transform(pil_image)
        image = image.unsqueeze(0)
        image = image.to(device)
        outputs = cnn(image)
        probabilities = F.softmax(outputs, dim=-1)
        predictions = torch.argmax(probabilities, dim=-1)

        mnist.utils.plot.generate_gradcam_class_plots(image, label,
        ↪ predictions, probabilities, cam, axes[i+1], base_image=False)

plt.tight_layout()
plt.show()

```



2 Trying a New Model and New Training Parameters

The performance of the network would likely improve with increased depth. The previous model's architecture is:

Input

```
-> Conv(1, 16, 3) -> ReLU -> BatchNorm(16)
-> Conv(16, 32, 3) -> ReLU -> MaxPool(2, 2)
-> Flatten -> Linear(6272, 10)
```

-> Output

where:

- Conv(x, y, z) is a Conv2d layer with x in channels, y out channels, kernel size of z, and zero padding to keep image dimensions the same,

- `ReLU` is a standard `ReLU` layer,
- `BatchNorm(x)` is a `BatchNorm2d` layer with `x` features,
- `MaxPool(x, y)` is a `MaxPool2d` layer with kernel size `x` and stride `y`,
- `Flatten` is a standard flatten layer that flattens tensors of shape `(B, C, H, W)` to `(B, C*H*W)`,
- `Linear(x, y)` is a `Linear` layer with `x` input features and `y` output features, and

The new architecture is significantly larger:

Input

```
-> Conv(1, 16, 3) -> ReLU -> BatchNorm(16)
-> Conv(16, 16, 3) -> ReLU -> MaxPool(2, 2) -> Dropout(0.25)
-> Conv(16, 32, 3) -> ReLU -> BatchNorm(32)
-> Conv(32, 32, 3) -> ReLU -> MaxPool(2, 2) -> Dropout(0.25)
-> Conv(32, 64, 3) -> ReLU -> BatchNorm(64)
-> Conv(64, 64, 3) -> ReLU -> MaxPool(2, 2) -> Dropout(0.25)
-> Flatten -> Dropout(0.5)
-> Linear(4096, 2048) -> ReLU -> Dropout
-> Linear(2048, 512) -> ReLU -> Dropout
-> Linear(512, 128) -> ReLU -> Dropout
-> Linear(128, 10)
```

-> Output

Note that, since `MaxPool2d(2, 2)` effectively reduces the dimensions of the tensor by a factor of 2, images can no longer be 28x28 pixels. This is because we have a total of three such layers, and $\frac{28}{2^3} = 3.5$ (not an integer). To fix this, we resize the images to 64x64 pixels before entering the network, allowing their width and height to be reduced more times.

Additionally, since the network is larger there is a larger opportunity for it to start overfitting on the training data. To remedy this, we add dropout layers, where `Dropout(x)` is a `Dropout` layer with a dropout probability of `x`.

In addition to the larger model, the training process is also being changed from the initial model.

- The model is trained longer (50 epochs instead of 30). This gives the model more time to train and allows the learning rate to decrease more gradually throughout the training loop.
- The training data transformation applies the maximum magnitude of transformations (rotations, translations, scale, and shear) immediately instead of gradually and keeps the this magnitude constant throughout training. This prevents the model from expecting digits to be centered and of similar size, instead forcing the model to learn the salient features of each digit.

```
[8]: from pathlib import Path
import matplotlib.pyplot as plt
import torch
from torchvision import transforms, datasets
import mnist
from mnist.model.train import train
```

```

epochs = 50
x_mean, x_std = (0.13066114485263824, 0.30810731649398804)
data_root = Path('../data')
batch_size = 64

if torch.cuda.is_available():
    device = 'cuda'
else:
    device = 'cpu'

print(f'Using {device}')

transform = transforms.Compose([
    transforms.Resize((64, 64)),
    transforms.RandomApply(
        [
            transforms.RandomAffine(
                degrees=(-30, 30),
                translate=(0.25, 0.25),
                scale=(0.5, 1.5),
                shear=(10, 10)
            )
        ],
        p=0.75
    ),
    transforms.ToTensor(),
    transforms.Normalize((x_mean,), (x_std,)) # (C, H, W)
])

train_dataset = datasets.MNIST(root=data_root, train=True, download=True,
    ↪transform=transform)
val_dataset = datasets.MNIST(root=data_root, train=False, download=True,
    ↪transform=transform)

train_loader = torch.utils.data.DataLoader(train_dataset,
    ↪batch_size=batch_size, shuffle=True)
val_loader = torch.utils.data.DataLoader(val_dataset, batch_size=batch_size,
    ↪shuffle=True)

cnn = mnist.model.cnn.CNN2(W=64, H=64)
cnn.to(device)

optimizer = torch.optim.AdamW(cnn.parameters(), lr=1e-4, weight_decay=0.01)
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=epochs,
    ↪eta_min=1e-6)

```



```
diagnostics = train(cnn, (train_loader, val_loader), epochs, optimizer, ↵
↵scheduler, flatten=False, device=device, quiet=True, show_progress=True)
```

Using cuda

100%| | 50/50 [09:51<00:00, 11.82s/it]

```
[9]: diagnostics.head()
```

```
[9]:
```

	Epoch	Learning Rate	Training Loss	Training Accuracy	Validation Loss	\
0	0	0.000100	2.303744	0.105469	2.303349	
1	1	0.000100	1.313342	0.594844	1.314987	
2	2	0.000100	0.617577	0.806094	0.607757	
3	3	0.000099	0.384654	0.880469	0.339368	
4	4	0.000098	0.230890	0.930937	0.225652	

	Validation Accuracy
0	0.103906
1	0.600938
2	0.807344
3	0.898125
4	0.933906

```
[10]: diagnostics.tail()
```

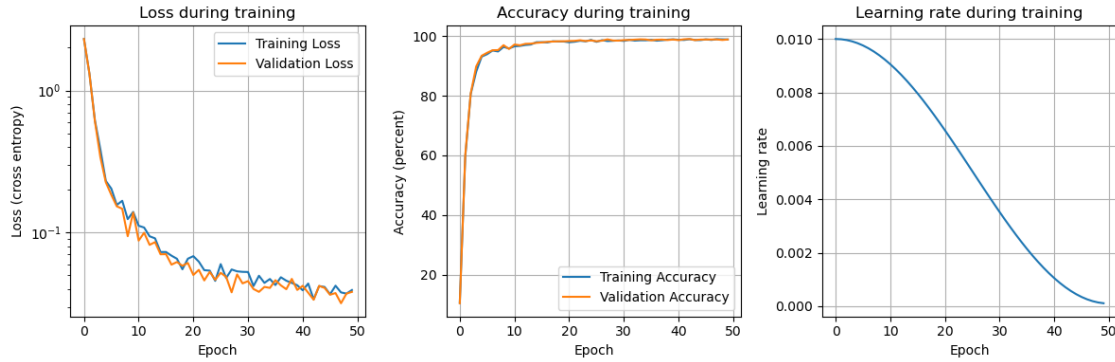
```
[10]:
```

	Epoch	Learning Rate	Training Loss	Training Accuracy	Validation Loss	\
45	45	0.000003	0.036788	0.988906	0.036516	
46	46	0.000003	0.042253	0.987031	0.037515	
47	47	0.000002	0.037922	0.989531	0.031811	
48	48	0.000001	0.037191	0.989062	0.037194	
49	49	0.000001	0.039407	0.988594	0.038249	

	Validation Accuracy
45	0.988437
46	0.988437
47	0.988750
48	0.986875
49	0.988437

This already looks promising, as the accuracy of both the training and validation sets reach close to 100%.

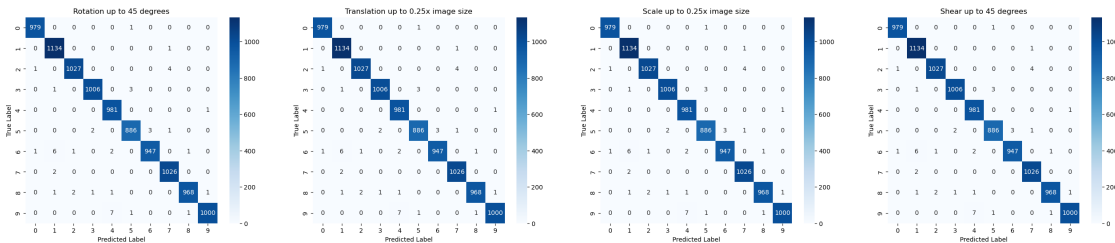
```
[ ]: fig: PltFigure
axes: PltAxes
fig, axes = plt.subplots(1, 3, figsize=(12, 4))
mnist.utils.plot.plot_diagnostics(diagnostics, axes)
plt.tight_layout()
plt.show()
```



3 Evaluating the Model

We use the same transformation confusions matrices visualizations as before, where we apply one transformation at a time and evaluate the model on the transformed data. The new model predicts almost all cases correctly for all transformations (rotations, translations, scaling, and shearing).

```
[ ]: fig: PltFigure
axes: PltAxes
fig, axes = plt.subplots(nrows=1, ncols=4, figsize=(32, 6))
mnist.utils.plot.generate_transformation_confusion_matrices(cnn, data_root,
    ↪ x_mean, x_std, axes, device, resize=(64, 64))
plt.show()
```



Now we evaluate the loss and accuracy of the model on the training, validation, and test datasets.

```
[18]: from mnist.model.train import dataset_loss
from torchvision import transforms, datasets
transform = transforms.Compose([
    transforms.Resize((64, 64)),
    transforms.ToTensor(), # (C, H, W)
    transforms.Normalize((x_mean,), (x_std,)) # (C, H, W)
])
val_dataset = datasets.MNIST(root=data_root, train=False, download=True,
    ↪ transform=transform)
```

```

batch_size = 64
val_loader = torch.utils.data.DataLoader(val_dataset, batch_size=batch_size,
    ↪shuffle=True)
estimated_loss = dataset_loss(cnn, val_loader, flatten=False, device=device)
print(f'Model achieved loss of {estimated_loss[0]:.4f} on the untransformed_
    ↪validation set (accuracy of {estimated_loss[1]*100:.2f}%).')

```

Model achieved loss of 0.0166 on the untransformed validation set (accuracy of 99.54%).

```

[19]: transform = transforms.Compose([
    transforms.Resize((64, 64)),
    transforms.RandomAffine(
        degrees=5,
        translate=(0.25, 0.25),
        scale=(0.75, 1.25),
        shear=5
    ),
    transforms.ToTensor(),                # (C, H, W)
    transforms.Normalize((x_mean,), (x_std,)) # (C, H, W)
])
val_dataset = datasets.MNIST(root=data_root, train=False, download=True,
    ↪transform=transform)
batch_size = 64
val_loader = torch.utils.data.DataLoader(val_dataset, batch_size=batch_size,
    ↪shuffle=True)
estimated_loss = dataset_loss(cnn, val_loader, flatten=False, device=device)
print(f'Model achieved loss of {estimated_loss[0]:.4f} on the transformed_
    ↪validation set (accuracy of {estimated_loss[1]*100:.2f}%).')

```

Model achieved loss of 0.0287 on the transformed validation set (accuracy of 99.27%).

Note that the test dataset are my own handwritten digits. Performing well on this dataset would indicate promising results for the kind of data the model would receive in the production environment.

```

[21]: from torchvision import transforms, datasets
test_data_root = Path('../custom-data')
x_mean, x_std = (0.13066114485263824, 0.30810731649398804)
transform = transforms.Compose([
    transforms.Resize((64, 64)),
    transforms.Grayscale(num_output_channels=1),
    transforms.ToTensor(),                # (C, H, W)
    transforms.Normalize((x_mean,), (x_std,)) # (C, H, W)
])
test_dataset = datasets.ImageFolder(root=test_data_root, transform=transform)

```

```
test_dataloader = torch.utils.data.DataLoader(test_dataset, batch_size=8,
↪shuffle=True)

images, labels = next(iter(test_dataloader))
mnist.utils.plot.show_images_with_labels(images, list(map(str, range(10))),
↪labels)
```



```
[22]: estimated_loss = dataset_loss(cnn, val_loader, flatten=False, device=device)
print(f'Model achieved loss of {estimated_loss[0]:.4f} on the test set,
↪(accuracy of {estimated_loss[1]*100:.2f}%).')
```

Model achieved loss of 0.0267 on the test set (accuracy of 99.26%).

4 Saving the Model

First we save the model as a PyTorch state dict for potential future debugging.

```
[23]: torch.save(cnn.state_dict(), './cnn2.pth')
print('Model saved')
```

Model saved

Then we save the model as ONNX to be used in the production environment.

```
[24]: cnn = mnist.model.cnn.CNN2(64, 64)
state_dict = torch.load('cnn2.pth')
cnn.load_state_dict(state_dict)
cnn.eval()
dummy_input = torch.randn(1, 1, 64, 64)
torch.onnx.export(
    cnn,
    dummy_input,
    'cnn2.onnx',
    export_params=True,
    opset_version=12,
    do_constant_folding=True,
    input_names=['input'],
    output_names=['output'],
```

```
dynamic_axes={'input': {0: 'batch_size'}, 'output': {0: 'batch_size'}}
)
```

/tmp/ipykernel_81700/318715524.py:2: FutureWarning: You are using `torch.load` with `weights\_only=False` (the current default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling (See <https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models> for more details). In a future release, the default value for `weights\_only` will be flipped to `True`. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be loaded via this mode unless they are explicitly allowlisted by the user via `torch.serialization.add\_safe\_globals`. We recommend you start setting `weights\_only=True` for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.

```
state_dict = torch.load('cnn2.pth')
```

Here is an example of how we would perform inference with the production ONNX model.

```
[25]: import torch
import onnxruntime as ort
from torchvision import transforms, datasets
from mnist.utils.plot import show_images_with_labels
import numpy as np

x_mean, x_std = (0.13066114485263824, 0.30810731649398804)

transform = transforms.Compose([
    transforms.Resize((64, 64)),
    transforms.ToTensor(),
    transforms.Normalize((x_mean,), (x_std,)) # (C, H, W)
])

transformed_train_dataset = datasets.MNIST(root=data_root, train=True,
    ↳download=True, transform=transform)
train_dataset_loader = torch.utils.data.DataLoader(transformed_train_dataset,
    ↳batch_size=4, shuffle=True)

session = ort.InferenceSession('cnn2.onnx')

inputs, labels = next(iter(train_dataset_loader))

outputs = session.run(None, {'input': inputs.numpy()})

predicted = np.argmax(outputs[0], axis=1) # Equivalent to `torch.max(outputs,
    ↳1)`
```

```
show_images_with_labels(inputs, list(map(str, range(10))), labels, predicted, ↵  
↪ 'ONNX')
```

ONNX

